

1. 개요

주어진 training set을 학습하는 과정에서 Gain Ratio를 통해 test attribute를 선정해 Decision tree를 구성하고, 만들어진 Decision tree를 통해 test set에 대한 classify를 진행한다. 이후 test set의 실제 class와 예측한 class를 비교해 정확도를 측정한다.

2. 알고리즘 요약

1) Information Gain / Gain Ratio

Test Attribute를 선택하기 위해 대표적으로 Information Gain을 사용한다. Entropy는 데이터가 얼마나 homogeneous 한지 나타내는 지표이며, homogeneous 하면 0이 된다.

여기서 특정 attribute를 기준으로 분할했을 때, 분할 후 엔트로피의 weighted average를 기존 엔트로피에서 뺀 값을 Information gain으로 사용한다. 분할 후 엔트로피가 작을수록 Gain이 커지므로, 가장 큰 Gain을 가지는 attribute를 test attribute로 선정한다.

$$Entropy(D) = - \sum_{i=1}^m p_i \log_2(p_i)$$
$$Gain(D, A) = Entropy(D) - \sum_{v \in \text{Values}(A)} \frac{|D_v|}{|D|} Entropy(D_v)$$

<Information Gain>

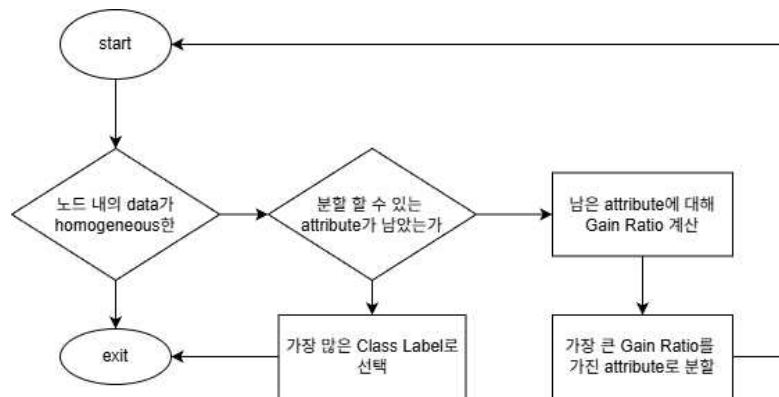
다만 Information Gain은 attribute가 multi-valued일 때 advantage를 주기 때문에 이를 해결하기 위해, Gain Ratio를 사용한다. Gain Ratio는 나눈 비율에 대한 엔트로피인 SplitInfo를 사용하여 value의 수가 많은 attribute에 패널티를 두어 test attribute를 공정하게 선정 할 수 있게 한다.

$$SplitInfo_A(D) = - \sum_{j=1}^k \frac{|D_j|}{|D|} \log_2 \left(\frac{|D_j|}{|D|} \right)$$
$$GainRatio(D, A) = \frac{Gain(D, A)}{SplitInfo_A(D)}$$

<Gain Ratio>

2) Decision Tree 구성 단계

Decision Tree는 모든 data가 루트 노드에서 시작해서, greedy하게 최적 test attribute를 선정해, 자식노드로 data를 분할한다. 이후 자식노드에 대해 재귀적으로 진행하여 최종적으로 노드의 data가 homogeneous하거나 더 이상 나눌 attribute가 없을 때까지 반복한다. 만약 더 이상 나눌 attribute가 없을 때에는, 남아 있는 데이터 내에서 가장 많은 Label로 클래스를 선택한다.



<그림 1> Decision Tree 구성 과정

3. 주요 함수 및 코드 설명

1) 트리 자료 구조

```

class Node:
    def __init__(self):
        self.attribute = None
        self.children = {}
        self.isLeaf = False
        self.classLabel = None
  
```

<그림 2> Node

트리를 구성하기 위해 노드를 정의한다. 분할을 위해 선택된 test attribute, 자식노드, 리프 노드 여부, 추정된 Class Label을 멤버로 갖는다.

2) getEntropy

```

def getEntropy(data):
    total = len(data)
    entropy = 0.0
    if total == 0:
        return entropy
    counts = {}
    for row in data:
        label = row[-1]
        counts[label] = counts.get(label, 0) + 1
    for count in counts.values():
        entropy -= (count / total) * math.log2(count / total)
    return entropy
  
```

<그림 3> getEntropy

데이터 내의 엔트로피를 구하기 위해 getEntropy 함수를 정의했다. 데이터 내에서 각 클래스인 마지막 attribute를 count해 전체에 대한 비율을 구해 계산한다.

3) buildTree

data에서 재귀적으로 decision tree를 구성하는 함수이다. 부모 노드로부터 내려온 데이터와 현재 노드까지의 경로에서 아직 test attribute로 사용되지 않은 attribute를 인자로 받는다.

먼저 flow chart에서 구상한대로, 종료 조건을 확인한다. 만약 label이 하나밖에 남지 않았다는 것은 현재 노드의 데이터가 homogeneous하다는 것을 의미하므로 그 클래스로 분류하고 노드를 반환한다. 또한 attribute의 개수가 1 이하라는 것은 마지막 attribute인 class label을 제외하고 더 이상 분할 attribute가 없다는 뜻으로 가장 많은 classLabel인 majorClass를 구해 선택한다. 개수가 같다면 알파벳 순으로 가장 앞의 클래스를 선택한다.

```
def buildTree(data, attributes):
    node = Node()
    labels = [row[-1] for row in data]

    if len(set(labels)) == 1:
        node.isLeaf = True
        node.classLabel = labels[0]
        return node

    majorClass = max(sorted(set(labels)), key=labels.count)

    if len(attributes) <= 1:
        node.isLeaf = True
        node.classLabel = majorClass
        return node
```

<그림 4> buildTree 분할 중단 조건

다음으로는 Test Attribute를 선정한다. 우선 현재 노드에서의 엔트로피를 구해 Base Entropy로 둔다. 이후 분할 후의 엔트로피를 구하는데, 명세에서 모든 attribute는 categorical이라고 주어졌기 때문에, attribute를 모든 value 값으로 나누어서 엔트로피를 구한다. 그리고 multi-valued attribute의 어드밴티지를 줄이기 위해 split Info로 나누어 Gain Ratio를 구한다. 다만 여기서 attribute value가 전부 같을 경우 split Info가 0이 되기 때문에 예외를 두었다. 만약 이렇게 구한 Gain Ratio가 기존보다 클 경우, 이를 업데이트 해준다.

모든 attribute를 살펴 본 후, 마지막으로 Gain Ratio가 큰 attribute를 test attribute로 선정한다. Class Label을 노드에서 가장 다수인 클래스로 두는데 이는 만약 attribute의 값이 Training set에는 없고, Test Set에만 있는 Unseen 상황에서 가장 많은 것으로 추측하도록 만들기 위함이다

```
baseEntropy = getEntropy(data)
GainRatio = -1.0
idx = -1

for i in range(len(attributes) - 1):
    attrValues = set([row[i] for row in data])
    tmpEntropy = 0.0
    splitInfo = 0.0

    for value in attrValues:
        subset = [row for row in data if row[i] == value]
        ratio = len(subset) / len(data)
        tmpEntropy += ratio * getEntropy(subset)
        if ratio > 0:
            splitInfo -= ratio * math.log2(ratio)
```

```
infoGain = baseEntropy - tmpEntropy

if splitInfo == 0:
    tmpGainRatio = infoGain
else:
    tmpGainRatio = infoGain / splitInfo

if tmpGainRatio > GainRatio:
    GainRatio = tmpGainRatio
    idx = i

node.attribute = attributes[idx]
node.classLabel = tmpclass
```

<그림 5> test attribute 선정

마지막으로 선정된 test attribute로 data를 분류해 자식 노드를 만든다. 먼저 attribute의 value마다 row를 모으고 선정된 attribute는 제외해 새로운 subset을 만든다. 그리고 attribute 리스트도 test attribute는 제외한다. 새로 생성된 subset과 attribute를 자식노드로 전달해 재귀적으로 decision tree를 완성한다.

```
attrValues = set([row[idx] for row in data])

for value in attrValues:
    subset = []
    for row in data:
        if row[idx] == value:
            newRow = row[:idx] + row[idx+1:]
            subset.append(newRow)

    if len(subset) == 0:
        leaf = Node()
        leaf.isLeaf = True
        leaf.classLabel = tmpClass
        node.children[value] = leaf
    else:
        newAttributes = attributes[:idx] + attributes[idx+1:]
        node.children[value] = buildTree(subset, newAttributes)

return node
```

<그림 6> child node 생성

4) classify

생성된 decision tree를 기반으로 test data를 받아 classify한다. 만약 리프 노드라면 노드의 Class Label로 추측하고, 아니라면 test attribute를 기준으로 자식노드를 추가 탐색한다. 예외 처리를 한 대로 만약 test set에만 있고 decision tree에는 없는 attribute value가 있다면 이는 decision tree를 만들 때 현재 노드에서 가장 많은 Class를 저장했으므로 Class Label로 추측한다.

```
def classify(node, testTuple, testAttributes):
    if node.isLeaf:
        return node.classLabel

    attrIdx = testAttributes.index(node.attribute)
    testValue = testTuple[attrIdx]

    if testValue not in node.children:
        return node.classLabel

    return classify(node.children[testValue], testTuple, testAttributes)
```

<그림 7> classify

5) main

먼저 training set 파일을 읽어 한 줄씩 파싱하여 data를 저장한다. 이후 decision Tree를 구성한다. 다음으로 test set을 열어 test tuple과 classify한 결과를 함께 output file에 출력한다.

```
def main():
    trainFile = sys.argv[1]
    testFile = sys.argv[2]
    outputFile = sys.argv[3]

    trainData = []
    trainAttributes = []

    with open(trainFile, 'r') as file:
        lines = file.readlines()
        if len(lines) > 0:
            trainAttributes = lines[0].strip().split('\t')
            for line in lines[1:]:
                items = line.strip().split('\t')
                trainData.append(items)

    root = buildTree(trainData, trainAttributes)
    testData = []
    testAttributes = []
    with open(testFile, 'r') as file:
        lines = file.readlines()
        if len(lines) > 0:
            testAttributes = lines[0].strip().split('\t')
            for line in lines[1:]:
                items = line.strip().split('\t')
                testData.append(items)

    with open(outputFile, 'w') as out:
        out.write('\t'.join(trainAttributes) + '\n')

        for testTuple in testData:
            predictedClass = classify(root, testTuple, testAttributes)

            result = testTuple + [predictedClass]
            out.write('\t'.join(result) + '\n')
```

<그림 8> main

buying	maint	doors	persons	lug_boot	safety	car_evaluation
med	vhigh	2	4	med	med	acc
low	high	4	4	small	low	unacc
high	vhigh	4	4	med	med	acc
high	vhigh	4	more	big	low	unacc
low	high	3	more	med	low	unacc
med	high	2	more	small	high	acc
vhigh	low	3	2	med	high	unacc
med	high	2	4	small	low	unacc
med	low	5more	4	small	med	good
med	low	5more	2	big	med	unacc
med	low	4	more	big	high	vgood
low	low	4	2	big	high	unacc
low	low	3	more	med	low	unacc
high	med	2	2	big	high	unacc

<그림 9> 출력 예시

4. 실행

1) 실행환경

OS : Window, Python Version : Python 3.12.2

2) 실행 방법

```
python dt.py dt_train1.txt dt_test1.txt dt_result1.txt
```

<그림 10> 실행 방법

dt.py파일이 있는 경로에서 python dt.py [training 파일] [test 파일] [출력 파일] 형식으로 실행하시면 됩니다. 기본적으로는 dt.py와 입출력파일이 같은 경로에 있는 것을 가정합니다.

3) 실행결과

```
buying maint doors persons lug_boot safety car_evaluation
med vhigh 2 4 med med acc
low high 4 4 small low unacc
high vhigh 4 4 med med acc
high vhigh 4 more big low unacc
low high 3 more med low unacc
med high 2 more small high acc
vhigh low 3 2 med high unacc
med high 2 4 small low unacc
med low 5more 4 small med good
med low 5more 2 big med unacc
med low 4 more big high vgood
low low 4 2 big high unacc
low low 3 more med low unacc
```

<그림 11> 출력 결과

4) 테스트

```
.\dt_test.exe dt_answer1.txt dt_result1.txt
```

제공된 dt_test.exe를 이용해 정확도를 평가한다.

```
PS C:\Users\HJ\
323 / 346
PS C:\Users\HJ\
```

<그림 12> 테스트 결과

decision tree로 분류한 결과 정답파일과 비교해 약 93%의 정확도를 보여준다.